

Angular 2025 – Domande e Risposte

Basato sul libro "Angular 2025 per Generation" – Capitoli 1–6

CAPITOLO 1 – Introduzione ad Angular

1. Cos'è Angular?

Angular è un framework front-end per la scrittura di Single Page Applications. È mantenuto da Google, si basa su TypeScript ed è tra le competenze più richieste nel settore. Risponde alla crescita della complessità delle applicazioni offrendo soluzioni standard ai problemi più comuni.

2. Quali sono le caratteristiche principali di Angular?

- Component-oriented: il front-end è scomposto in componenti indipendenti con logica e struttura proprie.
- TypeScript-based: sviluppo tramite TypeScript, superset di JavaScript con tipizzazione statica.
- Opinionated: fornisce soluzioni standard per API, form, testing — senza dipendere da librerie esterne.
- Dotato di Angular CLI (`ng`): utility da riga di comando per creare, configurare ed estendere progetti.

3. Cos'è la Angular CLI e come si installa?

È l'utility da linea di comando (`ng`) che automatizza i task più comuni. Si installa tramite npm:

```
npm install -g @angular/cli
```

4. Come si crea un nuovo progetto Angular?

```
ng new nomeProgetto
```

Viene creata una cartella con il nome del progetto contenente la struttura completa. La cartella `app` è quella in cui si lavora principalmente.

5. Cos'è un componente Angular?

Un componente è un pezzo tendenzialmente autonomo di interfaccia grafica, dotato di propria logica e gestibile separatamente. È composto da:

- `.ts` – la logica (classe TypeScript, il componente in senso stretto)
- `.html` – il template (parte visibile)
- `.css` – gli stili (applicati solo a quel componente)
- `.spec.ts` – i test automatizzati

6. Come si genera un nuovo componente?

```
ng generate component NomeComponente
```

Oppure in forma breve:

```
ng g c NomeComponente
```

7. Cosa contiene il decorator `@Component`?

- `selector`: il tag HTML con cui richiamare il componente (es. `<app-hello-world />`)
- `imports`: dipendenze del componente
- `templateUrl`: percorso del file HTML
- `styleUrl`: percorso del file CSS

8. Cos'è la sintassi di interpolazione `{{ }}`?

È la sintassi con cui si stampa un valore nel template. Calcola l'espressione tra le doppie parentesi graffe e la mostra a schermo. Es: `{{ message }}` stampa il valore della variabile `message` del componente.

9. Come si avvia il server di sviluppo Angular?

```
ng serve
```

Compila il progetto e lo rende disponibile su `http://localhost:4200`. Ogni modifica ai file viene rilevata automaticamente (watch mode).

10. Come si usa un componente in un altro componente?

Si inserisce il suo selettore nel template del componente padre (`<app-hello-world />`), si importa la sua classe nel decorator `@Component` padre tramite l'array `imports`, e si aggiunge l'import TypeScript in cima al file.

CAPITOLO 2 – Stato, Logica e Change Detection nell'era dei Signal

11. Cos'è lo "stato" di un componente?

Lo stato è l'insieme dei valori delle proprietà del componente in un dato momento. Può cambiare nel tempo, e spesso è la logica del componente stesso a occuparsi di questo cambiamento.

12. Come funziona la change detection classica di Angular e qual è il suo problema?

Con la change detection classica, ogni volta che lo stato del componente cambia, Angular ricalcola tutti i metodi e aggiorna l'intera vista. Su applicazioni grandi, questo ricalcolo indiscriminato può diventare costoso in termini di prestazioni (CPU, batteria, reattività).

13. Cos'è un Signal?

Un Signal è una funzione che "avvolge" una variabile di qualunque tipo e restituisce il suo ultimo valore. È in grado di notificare automaticamente i soggetti interessati (i suoi "observer") ogni volta che il suo valore cambia. Si crea con la funzione `signal(valoreIniziale)`.

14. Come si legge e si modifica il valore di un Signal?

- Lettura: si invoca come funzione — `italian()` restituisce il valore corrente.
- Scrittura con valore fisso: `italian.set(nuovoValore)`
- Scrittura basata sul valore precedente: `italian.update(v => v + delta)` — preferibile quando il nuovo valore dipende dal vecchio.

15. Perché usare `update()` invece di `set()`?

`update()` ricava sempre il nuovo valore dall'ultimo valore disponibile del signal, evitando problemi di "staleness" (obsolescenza). In scenari asincroni, `set()` potrebbe sovrascrivere un valore già aggiornato nel frattempo.

16. Cos'è un `computed` signal?

È un signal derivato che si ricalcola automaticamente solo quando uno dei signal da cui dipende cambia. Si definisce con:

```
techAverage = computed(() => (this.math() + this.programming()) / 2);
```

È anche **lazy** (si calcola solo quando viene letto) e **memoized** (memorizza il risultato finché le dipendenze non cambiano).

17. Cosa sono i side effect e come si gestiscono con `effect()`?

Un side effect è una reazione al cambiamento di un signal che non produce un valore (al contrario di `computed`). Si usa `effect()` per eseguire logica collaterale (chiamate API, localStorage, logging). Va dichiarato nel **costruttore** per avere un contesto di iniezione stabile:

```
constructor() { effect(() => { if (this.italian() == 10) alert('Complimenti!'); }); }
```

18. Quali sono i vantaggi dei Signal rispetto alla change detection classica?

- Dipendenze esplicite: solo i computed signal che dipendono da un dato signal vengono ricalcolati.
- Lazy evaluation: i computed vengono calcolati solo quando richiesti.
- Memoization: il risultato viene cachato finché le dipendenze non cambiano.
- Migliori prestazioni su applicazioni di medie e grandi dimensioni.

CAPITOLO 3 – Comunicazione fra Componenti

19. Cos'è il rapporto padre-figlio tra componenti Angular?

Quando un componente (padre) include il selettore di un altro componente (figlio) nel proprio template, si crea un rapporto di composizione. Il padre può avere più figli; ogni istanza figlio ha un solo padre diretto. Non è un rapporto di ereditarietà.

20. Come si passano dati dal padre al figlio (`input`)?

Nel figlio si dichiara una proprietà con `input()` o `input.required()`:

```
year = input.required<number>(); owner = input.required<string>();
```

Nel template del padre si usa il **property binding**:

```
<app-copyright [year]="currentYear" [owner]="currentOwner" />
```

21. Cosa significa `input.required()`?

Indica che il padre è obbligato a passare un valore per quel signal. In mancanza, si ottiene un errore in fase di compilazione. Il valore passato viene automaticamente avvolto in un signal read-only per il figlio.

22. Come si invoca un signal nel template?

Con le parentesi, come una funzione: `{{ year() }}`. I signal sono funzioni e vanno invocati per ottenerne il valore.

23. Cos'è il `@for` nel template Angular?

È una direttiva di controllo del flusso integrata nel motore di templating. Permette di scorrere una collezione e renderizzare un componente per ogni elemento:

```
@for (s of scores(); track s.subject) { <app-score [subject]="s.subject"
[value]="s.value" /> }
```

Il campo `track` è obbligatorio e serve a identificare univocamente ogni elemento per ottimizzare gli aggiornamenti.

24. Cos'è `model()` e quando si usa?

`model()` crea un signal bidirezionale (read-write) tra padre e figlio. A differenza di `input()` (read-only per il figlio), con `model()` il figlio può modificare il valore e il cambiamento si propaga automaticamente al padre. Si usa quando il figlio deve poter aggiornare un dato condiviso.

25. Cos'è la sintassi "banana in a box" `[(())]`?

È la sintassi di sincronizzazione bidirezionale. Unisce property binding `[ ]` (invia un valore) ed event binding `( )` (ascolta i cambiamenti). Si usa con `model()`:

```
<app-feature-score [(value)]="s.value" />
```

Si legge: "invia il valore di `value` al figlio e reagisci ai suoi cambiamenti".

26. Quali sono i tipi di binding nel template Angular?

- `{{ }}` – interpolazione (stampa un valore)
- `[ ]` – property binding (invia dati al figlio/elemento)
- `( )` – event binding (reagisce a un evento)
- `[( )]` – two-way binding / banana in a box (bidirezionale)

CAPITOLO 4 – Forms in Angular

27. Quali tipologie di form offre Angular?

1. **Template Driven Forms** – logica nel template, pensati per form semplici.
2. **Reactive Forms** – logica nel codice TypeScript, scalabili per form complesse.
3. **Signal Driven Forms** – sperimentali al momento della scrittura, non trattati nel corso.

28. Cos'è il Template Driven Form? Come funziona `ngModel`?

È un form gestito principalmente dal template. `ngModel` è il meccanismo chiave: collega una casella di testo a una proprietà del componente con two-way binding:

```
<input [(ngModel)]="weight" name="weight" type="number">
```

Questo richiede l'import di `FormsModule` nel componente.

29. Cosa sono le variabili di template (`#nomeVar`)?

Sono riferimenti a elementi o direttive Angular all'interno del template. Esempio:

`#bmiForm="ngForm"` crea un riferimento al form Angular, usabile per accedere al suo stato (es. `bmiForm.invalid`).

30. Come si gestisce la validazione nei Template Driven Forms?

Si usano attributi HTML standard (`required`, `min`, `max`) e si combinano con le variabili di template. Angular aggiorna automaticamente le proprietà `invalid`, `valid`, `touched`, `dirty`, `pristine` sui controlli e aggiunge classi CSS come `ng-invalid`, `ng-touched`:

```
@if (nameModel.invalid && nameModel.touched) { <small class="error">Valore obbligatorio.</small> }
```

31. Come si disabilita un pulsante di invio finché il form non è valido?

```
<button type="submit" [disabled]="recipeForm.invalid">Invia</button>
```

32. Cos'è un Reactive Form e come si crea?

È un form gestito dal codice TypeScript tramite un `FormGroup` (gruppo di `FormControl`). Si crea in `ngOnInit` con il `FormBuilder`:

```
constructor(private fb: FormBuilder) {} ngOnInit(): void { this.budgetForm = this.fb.group({ author: ['', Validators.required], email: ['', [Validators.required, Validators.email]], password: ['', [Validators.required, Validators.minLength(8)]] }); }
```

33. Cos'è un custom validator nei Reactive Forms?

È una funzione di validazione scritta dallo sviluppatore per logiche non coperte dai `Validators` standard. Riceve un `AbstractControl`, esegue la logica e restituisce un oggetto con gli errori trovati (es. `{ overBudget: true }`) oppure `null` se tutto è valido. Può essere applicato al singolo campo o all'intero form (cross-field validation).

34. Come si lega il template a un Reactive Form?

Con `[formGroup]` e `formControlName`:

```
<form [formGroup]="budgetForm" (ngSubmit)="onSubmit()"> <input type="text" formControlName="author"> </form>
```

35. Come si può integrare i Signal in un Template Driven Form?

Anziché usare `[(ngModel)]` (incompatibile con i signal), si separano property binding ed event binding:

```
<input [ngModel]="carbs()" (ngModelChange)="carbs.set($event)">
```

CAPITOLO 5 – Services

36. Cos'è un Service in Angular?

È codice condiviso fra più componenti e/o altri service, reso disponibile tramite Dependency Injection. Non ha un'interfaccia grafica. Gli utilizzi tipici sono: chiamate API, gestione dello stato condiviso, utility comuni.

37. Come si genera un service?

```
ng generate service NomeService
```

Oppure: `ng g s NomeService`. Produce solo file `.ts` (nessun template).

38. Cosa significa `@Injectable({ providedIn: 'root' })`?

Il decorator `@Injectable` rende la classe iniettabile tramite il sistema DI di Angular. `providedIn: 'root'` crea una singola istanza del service per l'intera applicazione (singleton), disponibile a qualsiasi componente o service che la richieda.

39. Come si inietta un service in un componente?

Tramite **constructor injection**:

```
constructor(private userService: UserService) {}
```

Oppure con la funzione `inject()`:

```
private http = inject(HttpClient);
```

40. Come si espone un signal in sola lettura da un service?

Con il metodo `.asReadonly()`:

```
private user = signal<User | null>(null); currentUser = this.user.asReadonly();
```

I componenti possono leggere `currentUser()` ma non modificarlo direttamente.

41. Cos'è `HttpClient` e come si usa in Angular?

È un service Angular per eseguire richieste HTTP, basato su RxJS. Prima di usarlo va abilitato in `app.config.ts`:

```
provideHttpClient()
```

Poi si inietta e si usa:

```
private http = inject(HttpClient); this.http.get<User[]>('users.json')
```

Il risultato è un `Observable`.

42. Cos'è un `Observable` e come si differenzia da un Signal?

Un `Observable` è un canale aperto su una sorgente di dati: può produrre valori nel tempo, notificare errori e il completamento. I subscriber si "abbonano" per ricevere i valori. Un `Signal` invece è sincrono e ha sempre un valore corrente. Si può convertire un `Observable` in `Signal` con `toSignal()`

43. Cos'è `toSignal()` e quando si usa?

Converte un Observable in un Signal: si abbona automaticamente, estrae il primo valore e lo mette a disposizione come signal. Richiede un valore iniziale (perché un signal deve sempre avere un valore):

```
private allUsers = toSignal( this.http.get<User[]>('users.json'),  
{ initialValue: [] } );
```

Deve essere chiamato in un **contesto di iniezione** (es. dichiarazione di variabile d'istanza o costruttore).

CAPITOLO 6 – Progetto Finale (Ticket Manager)

44. Qual è il progetto finale del corso?

Un'applicazione gestionale di ticket (richieste di intervento/segnalazioni) che include:

- Un back-end con Express + SQLite (`better-sqlite3`)
- Un front-end Angular con routing, componenti, service e chiamate API reali

45. Come si configura CORS nel back-end Express?

Si installa il modulo `cors` e si aggiunge come middleware:

```
const cors = require('cors'); app.use(cors());
```

Necessario perché il front-end Angular (porta 4200) e il back-end (porta diversa) girano su origini diverse.

46. Cos'è il Routing in Angular e come si configura?

Il routing permette di associare percorsi URL a componenti, realizzando la navigazione interna della Single Page Application senza ricaricare la pagina. Si configura in `app.routes.ts` e si abilita con `provideRouter(routes)` in `app.config.ts`.

47. Come si naviga tra le rotte in Angular?

- Nel template con la direttiva `routerLink`:

```
```html  
Vai ai ticket
```
```

- Nel codice TypeScript iniettando `Router` e usando `router.navigate(['/percorso'])`.

48. Come si leggono i parametri dalla URL in Angular?

Tramite `ActivatedRoute`:

```
private route = inject(ActivatedRoute); const id =  
this.route.snapshot.paramMap.get('id');
```

49. Come si gestisce lo stato di caricamento (loading) in un service con chiamate API?

Si usa un signal booleano che viene aggiornato prima e dopo la chiamata:

```
isLoading = signal(false); loadData() { this.isLoading.set(true);  
this.http.get(...).subscribe({ next: (data) => { /* ... */;  
this.isLoading.set(false); }, error: () => { this.isLoading.set(false); } });  
}
```

50. Qual è la differenza tra `@if` e `@for` nel template Angular?

- `@if (condizione) { ... }` – renderizza il blocco solo se la condizione è vera. Sostituisce la vecchia direttiva `*ngIf`.
- `@for (item of lista; track item.id) { ... }` – itera su una collezione e renderizza un blocco per ogni elemento. Sostituisce `*ngFor`. Il `track` è obbligatorio per l'identificazione univoca degli elementi.

RIEPILOGO CONCETTI CHIAVE

| Concetto | Descrizione |
|---------------------------|---|
| <code>signal(v)</code> | Crea un valore reattivo |
| <code>computed(fn)</code> | Valore derivato, ricalcolato solo se le dipendenze cambiano |
| <code>effect(fn)</code> | Side effect reattivo, eseguito al cambiamento di un signal |
| <code>input()</code> | Riceve dati dal padre (read-only per il figlio) |
| <code>model()</code> | Dati bidirezionali padre-figlio |
| <code>@Injectable</code> | Rende una classe iniettabile tramite DI |
| <code>FormGroup</code> | Gruppo di controlli di un Reactive Form |
| <code>toSignal()</code> | Converte un Observable in Signal |
| <code>HttpClient</code> | Service per chiamate HTTP, basato su RxJS |
| <code>routerLink</code> | Navigazione dichiarativa nel template |